



CockroachDB & Apache Cassandra

When and why each might be right for you and your workload

Apache Cassandra is a wildly successful open source project that has broad use as a “system of access” database for applications. It is a NoSQL database that is billed as “the right choice when you need scalability and high availability without compromising performance”. It scales linearly, tolerates node failures, and replicates data across multiple locations to reduce latency & improve resilience. It implements a “query first” architecture.

Apache Cassandra delivers some of the same value as CockroachDB, however, the choice between one or the other can be summarized in their architectural approach. Ultimately, it comes down to these key differences that define and differentiates the two solutions:

- **Apache Cassandra** was primarily designed to deliver fast reads and writes for very explicit, predetermined queries, largely in order to speed web apps.
- **CockroachDB** is architected primarily for high performant, consistent, distributed transactions and has substantial capabilities for distributed reads.

A good summary might be: *CockroachDB delivers a distributed foundation for effortless scale and resilience found in Apache Cassandra but with the familiarity, ease and power of PostgreSQL on top.* Choosing between them depends on your workload and what you need to accomplish.

Below we explore three key characteristics of each that may help you choose between them and then provide some insight into the architectural differences that define each.

Developer Experience - Using Standard SQL and Normalized Data

Apache Cassandra is extremely fast and incredibly powerful **if** you know exactly what you are looking for and **if** you put the data into the system the way you need to retrieve it. Most developers, however, find this experience difficult as it requires them to have insight into a very explicit form of the data in the database. It limits their ability to develop and execute ad-hoc queries.

When you use Cassandra for an application, your queries will define your data model, so you will need to know what you want to accomplish before you even start. Further, if you do choose to run a query for which the data model was not designed, then you'll have to create an additional copy of your table with keys designed for the new pattern, requiring the developer to ensure consistency between the two tables in the application tier.

Ultimately, you wind up with different copies of the data for each different query. (You'll have the same data with different primary key definitions). While delivering speed for accessible data, this tradeoff makes it difficult to maintain consistency between copies, and negatively impacts developer efficiency. Furthermore, upon writes, you now have to update in many places which can add further complexity because you may add an additional service layer to support updates of data across all tables. *Denormalization and duplication of data is a fact of life with Cassandra.*

This approach however is also extremely powerful as Cassandra eases the strictness of the guardrails on the middle layers of a database (indexes, partitions, etc) and gives you raw, unrestricted and high performant access to the data. You just need to know what you are doing and this is the tradeoff.

For most developers, however, this experience is limiting. They typically expect and appreciate the straightforward nature of SQL and the normalized form of data in a relational database. For experienced Cassandra developers they are comfortable and effective, but for the large majority it is a new paradigm, not relational SQL. CockroachDB

delivers familiar standard SQL for developers allowing them to be highly productive immediately.

Distributed Transactions - simple or complex?

CockroachDB implements Raft as its distributed consensus protocol, while Cassandra uses Paxos in only a limited way and does not leverage it as a consensus algorithm. Without diving into an academic exploration of the difference between the two, what is important, in this context, is that the simplicity of Raft allows CockroachDB to deliver complex, dynamic, multi-row transactions, while Cassandra is limited to simple straightforward, single row transactions.

There are no joins in the Cassandra Query Language (CQL) and the optimal goal would be to isolate all write operations on a single node. And while some have chosen to use Apache Spark (for analytics) or Apache Solr on top of Cassandra to facilitate joins, this adds another layer of software that further complicates development and management of the solution.

Denormalization and duplication of data is a fact of life with Cassandra. While you could work in more complex transactions, this would be quite a complex process and with CockroachDB is just part of the database. It delivers a normalized, relational database experience, complete with a query analysis tool and an advanced cost based optimizer.

Further, CockroachDB delivers serializable isolation (the highest level of transactional guarantee). Cassandra does not allow you to explicitly set an isolation level. Instead it relies on the concept of idempotency, which states that a transaction, once executed, can be executed multiple times without changing the result beyond the initial result. This allows the database to make key tradeoffs of consistency for speed. Additionally, Cassandra only provides transactional guarantees when explicitly requested within a 'lightweight transaction', and when configured to do so can only guarantee a transaction within a partition. Depending on application type, this might be important, but many disregard the risk associated with casual consistency delivered in most NoSQL databases. There is also no concept of a rollback in Cassandra. Thus, any write will eventually be completed regardless of if any of the nodes are available and the application developer is left to manage any discrepancy in consistency level if an operation is not executed correctly.

Control over global distribution

Apache Cassandra allows you to distribute data across broadly dispersed nodes, but you have to replicate data everywhere, which means you will need to replicate the data everywhere. If you have replication set to three, you will need three copies in each survivability zone. This not only adds bloat to the database (and subsequent cost), but it increases management complexity and can introduce consistency issues as all copies of data need to be updated in all regions. Ultimately, *denormalization and duplication of data is a fact of life with Cassandra*.

You can also tie data to a location using Cassandra but this is a complex process that requires an understanding of the data centers and replication factors in each. It can be complicated and requires a deeper understanding of Cassandra that a typical developer is not familiar with. One powerful use of isolation in Cassandra is their ability to isolate a workload on a set of data, so you could have different workloads working on the same data but in different data centers or availability zones.

CockroachDB provides a capability for geo-partitioning tables at the row level and setting and control of this is all managed by the database. It completely automates the movement of data and you can set survivability requirements without requiring any additional replicas. With geo-partitioning, you can tie data to any node or location within the cluster and ALTER this in production. This ability allows CockroachDB to meet SLAs for query latency and resilience and the ability to manage this as a simple ALTER TABLE command while in production provides massive control and tunability.

Architecture

While this document is not meant to be a dissertation or deep dive into the tech, we should explore at a very high level the architectural foundation of each of these databases. This is meant to provide context only and certainly a deeper investigation would reap a more complete and explicit comparison. A database can be divided into three primary components:

- *Access layer* - this defines how developers will interact with the database. It is the language used by our applications and services and often defined as SQL.
- *Execution layer* - this layer executes the incoming queries. While a transaction may appear to be simple in SQL syntax, often they get quite complex and this layer

executes requests to return data to our applications and services

- *Storage layer* - ultimately, data is written to disk and the methods used impact the capabilities of the database.

While it would be interesting to speak through Cassandra CQL v SQL or how transactions are executed, for this comparison, the base layer, the storage layer is of ultimate importance as it affects all of the aforementioned differences between Cassandra and CockroachDB. This layer defines how data is accessed/stored and in this case, how the data is distributed. This layer is where sharding is automated.

Both Cassandra and CockroachDB partition tables and distribute “chunks” of data across multiple nodes for storage within a cluster. How they partition is quite different, however.

Hash Partitioning v Range Partitioning

Cassandra uses hash partitioning to replicate and distribute data within a cluster. At a very high level, they take the keys for each row of data and apply a hash function to them so they can more easily manage them. *“A hash function is used to map data of arbitrary size to fixed-size values. The values returned by a hash function are called hash values, hash codes, digests, or simply hashes. The values are used to index a fixed-size table called a hash table. Use of a hash function to index a hash table is called hashing.” - wikipedia*

This is a rather broad simplification of how this works, but in Cassandra, every record has a primary key, which is comprised of a “partition key and “clustering columns”. A hash function applied to a “partition key”, which is used to determine on which node data will be stored. This hash is then used to quickly and easily identify where an explicit row of data lives among the distributed nodes and range scans become incredibly performant. Cassandra also uses a replication factor to write the data an explicit amount of times in a cluster and will distribute these replicas evenly across nodes within a cluster. There is greater complexity to how this is accomplished, but this high level will help us explain some of the benefits and challenges this presents.

As for transactions, this approach also limits some key capabilities.

- Cross partition transactions and multi-stage query processing is restricted and often handled using an additional tool, such as Apache Spark
- As consistency requirements increase, performance degrades via the CAP

theorem.

- You can set default consistency but this can be overridden by the developer/client on a per operation basis. This is often a daunting task to manage and small mistakes in development can cause large consistency problems in your applications.
- Clock skew in Cassandra used to be a problem and is often discussed but as long as you use NTP all writes will have a relatively consistent timestamp.

A bigger challenge for this approach is the addition of new nodes to a cluster and how it impacts existing data. With a new node, all hashes need to be recreated and the data redistributed, which causes impact in production. This is a great simplification but an area of concern that should be investigated.

CockroachDB uses a different approach called range partitioning to push data intelligently across nodes within a cluster. It is similar in many ways, but the underlying data does not need to know location and all data is lexicographically ordered. A deeper explanation of range partitioning can be found in our docs.

Which workload is right for which database?

As noted, workloads define the database and with Cassandra and CockroachDB, we can think about the following qualities of a workload to identify which database to use

	Single Data Center		Globally distributed	
Low read volume				
Heavy read volume and limited writes/transactions				
Heavy read, heavy write/update volume				
Heavy read/write volume and transaction guarantees				